

Data Editing Guide

Houston Rockets IT Team — Game Day Checklist

This guide covers how to add or remove tasks, create new sections, update the tech roster, and change the game schedule.

Files Referenced

File	Purpose
<code>checklists-full-setup.json</code>	Full game setup checklist
<code>checklists-partial-setup.json</code>	Partial game setup checklist
<code>checklists-full-breakdown.json</code>	Full game breakdown checklist
<code>checklists-partial-breakdown.json</code>	Partial game breakdown checklist
<code>techs.json</code>	Tech staff roster (dropdown names)
<code>Games.csv</code>	Game schedule (dates, times, opponents)

How the JSON is Structured

Before editing, it helps to understand the three levels:

- **Section** — a location or area (e.g. `preGame`, `DVOM`, `EastClub`)
- **Task** — a group of checklist items inside a section (e.g. `pg-1`, `pg-2`)
- **Item** — a single checkbox the tech checks off (e.g. `item-1`, `item-2`)

```
{  
  "preGame": {  
    "name": "FRONT TABLE/ BENCHES",  
    "techName": "(unassigned)",  
  },  
}
```

```

"tasks": [
  {
    "id": "pg-1",
    "title": "Table Connection",
    "subtitle": false,
    "assignedTech": "(unassigned)",
    "items": [
      {
        "id": "item-1",
        "label": "Your checklist item text goes here.",
        "completed": false
      }
    ],
    "managerVerified": false,
    "verifiedAt": null
  }
]
}

```

Note: All four checklist JSON files follow this exact same structure. Any change you make to one file must be manually repeated in the others if you want it reflected across all game types.

Adding a Task Item

Adding a new checkbox to an existing task

Find the task you want to update, then add a new object inside its **items** array. Make sure the **id** is the next number in sequence.

```

"items": [
  {
    "id": "item-1",
    "label": "Existing item.",
    "completed": false
  },
  {
    "id": "item-2",
    "label": "Your new checklist item goes here.",
    "completed": false
  }
]

```

```
}  
]
```

Adding a new task group to an existing section

Find the `tasks` array for the section you want, go to the end of it, and add a new task object. Increment the `id` — if the last task is `pg-3`, yours should be `pg-4`.

```
"tasks": [  
  { existing tasks above... },  
  {  
    "id": "pg-4",  
    "title": "Your New Task Title",  
    "subtitle": false,  
    "assignedTech": "(unassigned)",  
    "items": [  
      {  
        "id": "item-1",  
        "label": "First step for this task.",  
        "completed": false  
      }  
    ],  
    "managerVerified": false,  
    "verifiedAt": null  
  }  
]
```

Important: Always set `completed` to `false`, `managerVerified` to `false`, and `verifiedAt` to `null` when adding new tasks. These are the default starting values for every game day.

Removing a Task

Removing a single checkbox item

Find the item inside the `items` array and delete the entire object from the opening `{` to the closing `}`. Make sure to also remove the comma before or after it so the JSON stays valid.

Removing an entire task group

Delete the entire task object from the `tasks` array. Same rule applies — watch the trailing comma on the item before it.

Tip: After making edits, paste your file into jsonlint.com to check for errors before saving. A single misplaced comma will break the app.

Note: You do not need to renumber IDs after deleting. Leaving gaps (e.g. `pg-1`, `pg-3` with no `pg-2`) is fine — the app uses them as unique identifiers, not as a display order.

Creating a New Section

A section is a top-level key in the JSON file (like `preGame`, `DVOM`, `EastClub`). To add a new one, scroll to the bottom of the file just before the final closing `}` and add your new section.

```
{
  "preGame": { ... },
  "DVOM": { ... },
  "EastClub": { ... },

  "PressRow": {
    "name": "PRESS ROW CHECKLIST",
    "techName": "(unassigned)",
    "tasks": [
      {
        "id": "pg-1",
        "title": "Setup Checklist",
        "subtitle": false,
        "assignedTech": "(unassigned)",
        "items": [
          {
            "id": "item-1",
            "label": "Your first task item here.",
            "completed": false
          }
        ],
        "managerVerified": false,
        "verifiedAt": null
      }
    ]
  }
}
```

Rules for new sections:

1. The key name (e.g. "PressRow") is used internally — keep it short with no spaces or special characters.
2. The "name" field is what displays in the app — use all caps to match the existing style.
3. Start task IDs at "pg-1" and item IDs at "item-1" — these are scoped to their own section so there's no conflict with others.
4. If the section applies to both setup and breakdown, add it to all relevant files.

Important: After adding a new section to the JSON, a developer will also need to update `App.js` in the frontend to register the new tab in the UI. Adding it to the JSON alone will not make it appear in the app.

Adding or Removing a Tech

The tech roster lives in `techs.json`. This controls who appears in the assignment dropdown in the app.

The entire file is a single array of names:

```
{
  "techOptions": [
    "Brandon Searcy",
    "Carolyn Jopkins",
    "Darell Clark",
    "Eduardo Sosa",
    "Kiersten Bray",
    "NeQuarnda Dawson",
    "Bryson Cole",
    "Sinclair Hoyt"
  ]
}
```

- **To add a tech:** Add their name as a new string inside the array, with a comma after the previous entry.
- **To remove a tech:** Delete their name and the comma before or after it.
- **Name format:** Use "First Last" exactly as you want it to appear in the dropdown.
- **Order:** The order of names in the array is the order they appear in the dropdown.

Updating the Game Schedule

The game schedule lives in `Games.csv`. The app reads this file to display the current or next upcoming game. It automatically picks the nearest future game by date.

The file has four columns:

Column	Format	Example
<code>date</code>	YYYY-MM-DD	<code>2026-10-21</code>
<code>time</code>	H:MM TZ	<code>7:00 CST</code>
<code>opponent</code>	Text	<code>Rockets vs. Golden State Warriors</code>
<code>managerName</code>	Text	<code>Jennifer H. and Derrick M.</code>

Critical: Dates must always use the `YYYY-MM-DD` format. The app compares dates as text strings — any other format will break the sorting logic and the wrong game will display.

Adding new games

1. Open `Games.csv` in Excel, Google Sheets, or a text editor.
2. Do not change the header row — it must stay exactly as:
`date,time,opponent,managerName`
3. Add one row per game following the existing format.
4. Use `CST` for games before the clock change and `CDT` for games after.
5. Keep rows sorted by date (oldest to newest) for easier maintenance.
6. If editing in Excel, save as `.csv` — not `.xlsx`. The backend only reads plain CSV files.
7. Commit and push the updated file to GitHub. Render will redeploy automatically.

Changing the manager name

Update the `managerName` column for whichever games apply. You can list multiple names — separate them with `and` as currently done.

How the app picks the current game

The app filters for games with a date on or after today, sorts them, and displays the first one. If all dates are in the past, it falls back to the last game in the file. Always keep upcoming game dates in the file so the correct game loads automatically.

*For questions or issues, open a GitHub Issue at
github.com/RocketsITTeam/Game-Day-Checklist/issues*